Machine Learning Evaluation of Generative AI Debugging Feedback Plausibility for Introductory Python Programming

Brendan Toscano
Acadia University, Canada
0301485t@acadiau.ca

Lydia Bouzar-Benlabiod
Acadia University, Canada
lydia.bouzar-benlabiod@acadiau.ca

Darcy Benoit
Acadia University, Canada
darcy.benoit@acadiau.ca

***Abstract*—Large language models can often repair beginner programming errors, but a correct repair does not necessarily come with helpful feedback. This paper investigates whether the plausibility of LLM-generated debugging feedback can be estimated using computable features derived from existing fields in the IntroPyNUS dataset extended by Hints-in-Browser. We cleaned and flattened 10,548 records of buggy student submissions, repaired code, hints, and explanations across five introductory Python problems. From this data, we defined features for repair correctness, feedback relevance, hint revealingness, and repair minimality, and used them to construct both binary and multi-class plausibility labels. We then trained evaluator models on Problems 1 through 4 and tested them on held-out Problem 5. The best binary evaluator, a five-feature Linear SVM, achieved 92% accuracy and an F1 score of 0.76 on the plausible class. The three-feature Random Forest reached 86.3% accuracy and a macro F1 of 0.876 in the multi-class scheme using only `PRelevance`, `HRevealingness`, and `CDistance`, without direct access to the `PassFlag` repair-correctness feature.**

***Index Terms*—programming education, large language models, generative AI, debugging feedback, plausibility scoring, machine learning evaluation**

I. Introduction

Generative large language models (LLMs) are increasingly used to generate hints, repair suggestions, and natural-language explanations for beginners learning programming [1]–[3]. However, evaluating the feedback that accompanies a repair, especially at scale, is more difficult than evaluating the repaired code itself. Automated code repair can be measured with unit tests, which show whether the program passes but not whether the surrounding hint helps the student understand the bug. A repaired program can pass every unit test and still teach a beginner very little.

This paper addresses these concerns by developing an automated evaluation pipeline for LLM-generated debugging feedback. Rather than adding new human annotations or using another LLM as a judge, we compute response-level features from the IntroPyNUS dataset as extended by Hints-in-Browser [3], [4]. The goal is not to replace human evaluation, but to test whether reproducible features can estimate feedback plausibility while avoiding LLM-as-judge methods, which recent work has criticized on validity and reliability grounds [5].

This paper makes four contributions:

- **Dataset processing:** We restructure the extended IntroPyNUS dataset into a response-level format for feature computation and evaluator training.
- **Computed features:** We define three deterministic features, `PRelevance`, `HRevealingness`, and `CDistance`, that approximate feedback grounding, hint revealingness, and repair minimality.
- **Plausibility schemes:** We construct a strict binary `PlausibilityA` label and a multi-class `PlausibilityB` label using training-derived quantile buckets.
- **Plausibility Validation and Generalization Test:** We train evaluators on Problems 1-4 and test them on held-out Problem 5, using feature configurations that compare feedback-quality features with repair-correctness features.

The remainder of the paper is organized as follows. Section II reviews related work and the hint-quality rubric. Section III describes the dataset and tokenizer. Sections IV and V present the computed features and plausibility schemes. Section VI reports the evaluator results, Section VII discusses limitations, and Section VIII concludes the paper.

II. Related Work

Many evaluations of LLM-generated programming feedback rely on unit-test pass rate as the primary metric [6]. Benchmarks such as HumanEval [6], MBPP [7], SWE-bench [9], and other educational program-repair benchmarks [10] show that modern LLMs can synthesize or repair code, but their primary evaluation metric remains functional correctness. These benchmarks do not measure whether a hint is grounded in the student's bug, whether it reveals too much, or whether the repair stays close to the student's original code. For programming feedback, correctness is necessary but not sufficient.

The IntroPyNUS dataset [4] provides buggy and fixed Python submissions from an introductory programming course at the National University of Singapore. Hints-in-Browser [3] extends this dataset by attaching LLM-generated hints, explanations, and repaired code to each buggy program, along with a token-level edit-distance value and a binary repair-correctness flag. This extended dataset forms the basis of our study. Our goal is to determine whether deterministic response-

level features can provide a reproducible indication of feedback plausibility without requiring new human annotations.

One alternative to human evaluation is to use an LLM as a judge. LLM-as-judge systems such as G-Eval and MT-Bench/Chatbot Arena show why this approach is attractive [11], [12], but Chehbouni et al. [5] argue that the validity and reliability of such protocols have not been scrutinized in proportion to their adoption.

The computed features operationalize qualities highlighted in prior work on programming feedback. Phung et al. [1], [2], [8] and Hints-in-Browser [3] evaluate hint quality using four expert-rated binary attributes: `HCorrect`, `HInformative`, `HConceal`, and `HComprehensible`. A hint is considered good only when all four attributes are satisfied. Hints-in-Browser also evaluates repair quality using `RPass`, which indicates whether candidate repairs pass the task test suite, and `REdit`, the token-based edit distance between the repaired and buggy programs. This paper builds deterministic proxies for the parts of these criteria that can be estimated from response-level fields, rather than attempting to replicate expert-assigned hint labels.

To address these concerns, we compute features deterministically from the dataset itself. The plausibility labels are constructed from existing fields and a simple tokenizer, and no model judgment enters the labels or evaluator inputs. This approach does not resolve the deeper question of whether the features are strong proxies for hint quality, but it keeps the measurement reproducible. `PRelevance` targets `HInformative` through set-based token overlap between feedback and local problem context; `HRevealingness` inverts `HConceal` through token-frequency overlap between the hint and repaired code; and `CDistance` targets repair minimality by comparing the candidate's edit distance with the canonical edit distance for the same buggy submission. These proxies do not directly cover `HCorrect` or `HComprehensible`, and they are not equivalent to expert ratings. Instead, they provide a reproducible measure for estimating feedback plausibility without new human annotation.

III. Dataset and Tokenization

A. The Dataset

This paper uses the Hints-in-Browser extension of the IntroPyNUS dataset, which comprises five introductory Python programming problems. Each buggy submission is associated with generated repaired code, a hint, an explanation, a repair-correctness flag, and a token-level edit-distance value. The extended dataset has a nested structure in which each buggy program contains one canonical response and a list of alternate generated responses. In total, the dataset contains 1,758 buggy submissions, each with one canonical response and five alternate responses, yielding 1,758 canonical responses and 8,790 alternate responses. Table II reports the per-problem statistics after cleaning and flattening, including response-level row counts, pass rates, and counts of rows with valid `CDistance` values.

B. Tokenizer

a) Hints-in-Browser code tokenizer: We use the Hints-in-Browser token-level comparison method for `EditDistance` and `MinDistance`. Here, `MinDistance` denotes the canonical edit distance for the same buggy submission. This method uses Pygments `PythonLexer` tokenization and Levenshtein distance. It is separate from the overlap tokenizer described below.

b) Our overlap tokenizer: The overlap-based features use a simple tokenizer that lowercases natural-language feedback, problem text, buggy code, and repaired code before extracting word-like tokens. It also separates `camelCase` and `snake_case` variable names so that their components can contribute separately to the overlap token calculations. These tokens are used to compute lexical overlap for feedback relevance and hint revealingness in the next section.

C. Dataset Limitations

The dataset has several limitations. First, the canonical response is chosen based on correctness and edit distance, biasing the dataset toward minimal, functionally correct repairs. This selection does not account for cases in which a different correct repair may also be useful for learning. Second, the repair-correctness flag is based on fixed unit tests, which verify functional behavior but do not assess whether the repair is pedagogically useful, minimal, or clearly explained. Finally, because all hints, explanations, and repairs in the dataset were generated by the same GPT-4 model, the feedback reflects a single model style rather than a broader range of human tutoring styles.

IV. Pre-processing and Computed Features

A. Pre-processing the Dataset

We clean and flatten the nested structure so that each flattened row represents one generated response for one buggy submission. The resulting table contains 10,548 response-level rows. It preserves `ProblemId` and adds fields such as `AttemptId` and `ResponseKind` to track the original nested structure. We also add the train-test split used for ML evaluation: Problems 1-4 are used for training, and Problem 5 is held out for testing. This split yields 9,930 training rows and 618 held-out test rows before subsequent feature filtering. Splitting by problem reduces leakage among rows that share the same problem statement, function names, expected solution pattern, and bug structure.

B. Computed Features

We define three core features for constructing plausibility labels: `PRelevance`, `HRevealingness`, and `CDistance`. These features are computed from existing dataset fields and information derived for each response row. The label construction also uses `PassFlag`, a standardized binary version of the dataset's `correct_repair` field, along with thresholds, normalization, and quantile rules.

TABLE I
HELD-OUT PROBLEM 5 EVALUATOR RESULTS. FOR PLAUSIBILITYA, F1 IS REPORTED ON CLASS 1; FOR PLAUSIBILITYB, F1 IS MACRO-AVERAGED.

Scheme	Features	Model	Best parameter	CV F1	Accuracy	Test F1
A	Base (3)	Logistic Regression	$C = 100$	0.803	0.90	0.69
A	Base (3)	Linear SVM	$C = 0.1$	0.803	0.91	0.73
A	Extended (5)	Logistic Regression	$C = 10$	0.860	0.92	0.74
A	Extended (5)	Linear SVM	$C = 1$	0.861	0.92	0.76
B	Base (3)	Random Forest	depth = 10, trees = 200	0.801	0.863	0.876
B	Extended (5)	Random Forest	depth = None, trees = 200	0.989	0.998	0.997

a) PassFlag: This feature records whether the repaired program passes all fixed unit tests for the corresponding problem. It does not assess the quality of the accompanying hint or explanation. It is derived from `correct_repair` and stored as a consistent binary value. Although `PassFlag` is not a direct measure of tutoring quality, it provides a hard correctness signal.

b) PRelevance: This feature is a lexical grounding proxy between the feedback text and the local problem context. It compares the feedback text, defined as the hint plus explanation, with the local context, defined as the `llmDescription` mapped from `ProblemId` plus the buggy code. `PRelevance` is computed as the proportion of unique feedback tokens that also appear in the local context, preventing repeated words from inflating the score.

Let P, B, H, and E denote the problem text, buggy code, hint, and explanation, respectively. Let $\oplus$ denote string concatenation with a separating space.

$PAC = \text{set}(\text{tokenize}(P \oplus B))$ and $HAE = \text{set}(\text{tokenize}(H \oplus E))$.

$$\text{PRelevance} = \begin{cases} 0, & |HAE| = 0, \\ \dfrac{|PAC \cap HAE|}{|HAE|}, & \text{otherwise.} \end{cases} \quad (1)$$

c) HRevealingness: This feature estimates how much of the repaired code appears in the hint. Higher values of `HRevealingness` are treated as a negative signal because they suggest that the hint may reveal too much of the solution. It is computed as the overlap between hint tokens and repaired-code tokens, divided by the total number of repaired-code tokens. Thus, `HRevealingness` approximates the inverse of hint concealment.

Let H and R denote the hint and repaired code respectively. Let $H_T = \text{tokenize}(H)$ and $R_T = \text{tokenize}(R)$.

Let $O(H_T, R_T) = \sum_t \min(HC(t), RC(t))$, where $HC(t)$ and $RC(t)$ are token counts in H_T and R_T.

$$\text{HRevealingness} = \begin{cases} 0, & |R_T| = 0, \\ \dfrac{O(H_T, R_T)}{|R_T|}, & \text{otherwise.} \end{cases} \quad (2)$$

d) CDistance: This feature compares a candidate repair's edit distance with the canonical edit distance for the same buggy submission. `CDistance` is defined as a ratio relative to the canonical minimum edit distance, so higher values indicate that the candidate repair is closer to the canonical minimal fix in edit-distance terms. Rows without valid candidate or canonical edit distances cannot receive a `CDistance` value.

Let CED be the canonical edit distance and ED be the candidate edit distance.

Let CD be $\dfrac{CED}{\max(ED, 1)}$

$$\text{CDistance} = \begin{cases} \text{Invalid}, & \text{NaN } CED \text{ or } ED, \\ 1, & \min(CED, ED) \leq 0, \\ \min(1, CD), & \text{otherwise.} \end{cases} \quad (3)$$

C. Normalization and Edge Cases

`PRelevance` is set to 0 when the feedback text has no valid tokens, and `HRevealingness` is set to 0 when the repaired code has no valid tokens. `CDistance` is treated as missing when either the candidate edit distance or the canonical minimum edit distance is invalid. After the problem-level train-test split, rows with missing `CDistance` are excluded from evaluator training and testing, leaving 7,338 training rows and 444 held-out test rows.

V. PLAUSIBILITY SCHEMES

We use the computed features to construct two target labels for feedback plausibility. In the absence of human ratings of hint quality, these deterministic features support predictions of overall feedback quality. We define two plausibility labeling schemes: `PlausibilityA` provides a binary baseline, while `PlausibilityB` separates passing feedback into multiple levels. The thresholds were selected using a custom rubric, manual inspection of borderline cases, and the training distribution, informed by prior hint-quality criteria from previous studies [1]–[3]. The thresholds are not intended to represent universal cutoffs for hint quality; rather, they provide a reproducible basis for testing whether constructed plausibility labels can be predicted from deterministic features.

A. PlausibilityA: binary classification

`PlausibilityA` acts as the binary target. When `PassFlag` = 1, `PRelevance` ≥ 0.25, `HRevealingness` ≤ 0.15, and `CDistance` ≥ 0.80, then `PlausibilityA` is 1. If any of these conditions are not met, `PlausibilityA` is assigned 0. Prior hint-quality rubrics commonly use binary criteria for individual quality attributes [1]–[3], but this makes

PlausibilityA a strict baseline: a response that narrowly misses one threshold receives the same label as a response that fails every condition.

B. PlausibilityB: multi-class classification

To help capture rows that narrowly miss one of the thresholds in PlausibilityA, we define PlausibilityB. PassFlag remains a hard condition for PlausibilityB, but the three feature thresholds are used as normalization baselines rather than final cutoffs. PRelevance, inverted HRevealingness, and CDistance are normalized and averaged into a continuous ScoreB. This score is then assigned to ordered quantile buckets computed from the training split. PlausibilityB has four classes: 0, 1, 2, and 3. Among rows with valid feature values, class 0 represents failed repairs, while classes 1, 2, and 3 represent the lower, middle, and upper buckets among passing responses. Rows with missing CDistance are excluded before evaluator training and testing. PlausibilityB uses the following baseline thresholds before normalizing each feature to a $[0, 1]$ range:

- $\texttt{PRelMinB} = 0.25$
- $\texttt{HRevealMaxB} = 0.15$
- $\texttt{CDistMinB} = 0.80$

Normalized relevance:

$$\texttt{normPrel} = \text{clip}\left(\frac{\texttt{PRel} - \texttt{PRelMinB}}{1 - \texttt{PRelMinB}}, 0, 1\right)$$

Normalized concealment (inverted):

$$\texttt{normReveal} = \text{clip}\left(\frac{\texttt{HRevealMaxB} - \texttt{HReveal}}{\texttt{HRevealMaxB}}, 0, 1\right)$$

Normalized edit distance similarity:

$$\texttt{normCDist} = \text{clip}\left(\frac{\texttt{CDist} - \texttt{CDistMinB}}{1 - \texttt{CDistMinB}}, 0, 1\right)$$

Here, $\text{clip}(x, 0, 1)$ truncates values below 0 to 0 and values above 1 to 1.

$$\text{ScoreB} = \frac{\text{normPrel} + \text{normReveal} + \text{normCDist}}{3}. \quad (4)$$

Only passing rows in the training split are used to compute the quantile boundaries. The resulting boundaries were $q_1 = 0.333$ and $q_2 = 0.415$.

$$\text{PlausibilityB} = \begin{cases} 0, & \text{PassFlag} = 0, \\ 1, & \text{ScoreB} \leq 0.333, \\ 2, & 0.333 < \text{ScoreB} \leq 0.415, \\ 3, & \text{ScoreB} > 0.415. \end{cases}$$

VI. ML Evaluators and Results

This section reports the performance of the ML evaluators on the 444-row Problem 5 test set. The evaluators are not intended to discover independent ground truth because the plausibility labels are constructed deterministically; they test whether the constructed screening rules can be approximated from selected feature subsets under a held-out problem split. We present results for both classification types: PlausibilityA (binary) and PlausibilityB (multi-class). Two feature configurations are compared for each. The base model uses only the three computed feedback-quality features; the extended model adds PassFlag and EditDistance. The extended model reveals how much of the plausibility signal depends on feedback quality versus repair correctness and serves as an ablation.

A. Feature sets and model pairings

The model predicts the binary PlausibilityA using Logistic Regression and Linear SVM classifiers. For each configuration, we tuned the regularization C and the decision threshold via cross-validation on the training split. Once trained, the model is evaluated on the 444-row Problem 5 test set.

For PlausibilityB the model predicts the four-class target label using a Random Forest classifier with GridSearchCV over n_estimators and max_depth. The evaluation metric is macro-averaged F1, which treats each class equally regardless of its frequency.

B. Train/Test split

Problems 1-4 are used for training and Problem 5 for testing. Rows with missing values in the required feature columns are removed. After dropping incomplete rows, the effective training set contains 7,338 rows and the test set contains 444 rows. All classifier results below use the filtered set of 7,338 training rows and 444 test rows, since every feature must be present for a row to be scored. The test-label distributions are 387/57 for PlausibilityA and 171/109/53/111 for PlausibilityB; majority baselines are 87.2% accuracy with 0.00 class-1 F1 and 38.5% accuracy with 0.139 macro F1, respectively.

C. PlausibilityA: Binary classification

The base feature set consists of PRelevance, HRevealingness, and CDistance. These are the three computed features that correspond directly to the qualities used in the definition of the plausibility threshold, with the exception of PassFlag. The model is expected to determine plausibility from the proxy characteristics available without PassFlag.

Logistic Regression selected a regularization of C = 100 via cross-validation, achieving a Best CV F1 of 0.803 on the minority class. On the test set at the tuned threshold, the model achieved 90% accuracy with a class-1 F1 score of 0.69. Linear SVM selected C = 0.1 and achieved the same CV F1 of 0.803. On the test set, the tuned SVM reached 91% accuracy and a class-1 F1 score of 0.73.

Linear SVM with five features selected C = 1 and achieved a CV F1 score of 0.861, the highest across all binary configurations. At the tuned threshold, the SVM achieved 92% accuracy and a class-1 F1 score of 0.76. The SVM caught 54 of 57

TABLE II
PER-PROBLEM DATASET STATISTICS AFTER CLEANING AND FLATTENING THE INTROPYNUS HINTS-IN-BROWSER DATASET.

Problem	Task	Buggy subs.	Rows	PassFlag=1	Pass rate	Fail rate	Valid `CDistance`
1	Sequential Search	570	3,420	2,813	82.3%	17.7%	3,318
2	Unique Dates and Months	430	2,580	1,823	70.7%	29.3%	2,322
3	Duplicate Elimination	303	1,818	427	23.5%	76.5%	732
4	Sorting Tuples	352	2,112	508	24.1%	75.9%	966
Train total (Problems 1-4)		1,655	9,930	5,571	56.1%	43.9%	7,338
5	Top-k Elements	103	618	273	44.2%	55.8%	444
Total (Problems 1–5)		1,758	10,548	5,844	55.4%	44.6%	7,782

Note. Rows are response-level records. Each buggy submission has one canonical response and five alternate responses. `PassFlag=1` counts rows whose repaired code passes the unit tests. Valid `CDistance` counts rows retained after removing rows with missing candidate or canonical edit-distance values.

plausible rows, missing only 3. There were 31 false positives, the lowest count among all `PlausibilityA` models at their tuned thresholds.

Despite the gains from the extended feature set, the three-feature base model still provides a reasonable baseline; both Logistic Regression and Linear SVM achieved CV F1 scores of 0.803 using only `PRelevance`, `HRevealingness`, and `CDistance`. The performance of the five-feature models is improved on every metric, confirming that `PassFlag` and `EditDistance` carry strong predictive weight. This suggests that the base features alone provide useful but incomplete information, and that repair correctness still remains the dominant factor in determining plausibility.

D. PlausibilityB: Multi-class classification

The Random Forest with three base features selected `max_depth` = 10 and `n_estimators` = 200, achieving a CV macro F1 score of 0.801. On the test set, the model reached 86.3% accuracy and a macro F1 score of 0.876. Its test-set confusion matrix, with actual classes as rows and predicted classes as columns, was:

	0	1	2	3
0	139	25	4	3
1	26	83	0	0
2	0	1	52	0
3	0	0	2	109

Most errors occur at the class 0/class 1 boundary; classes 2 and 3 are nearly diagonal. The fitted model's feature importances were `CDistance`=0.443, `HRevealingness`=0.410, and `PRelevance`=0.147.

Adding `PassFlag` and `EditDistance`, the Random Forest selected `max_depth` = None and `n_estimators` = 200, yielding a CV macro F1 score of 0.989. On the test set, the model achieved 99.8% accuracy and a macro F1 of 0.997. This near-perfect result should be interpreted mainly as a reconstruction of the constructed label, since `PassFlag` directly separates class 0 from the passing classes.

E. Base vs extended feature comparison

Table I summarizes the main evaluator results on the held-out Problem 5 test set. Across both classification schemes, the extended feature set outperforms the base set on every metric. This is because the extended set includes the binary repair-correctness value and `EditDistance`, both of which are closely related to the constructed plausibility labels.

The base models are more constrained; they use only `PRelevance`, `HRevealingness`, and `CDistance` to show whether the computed feedback-quality features carry signal without directly using the binary repair-correctness value. The comparison between the base and extended models is therefore used to separate feedback-quality signal from correctness-related signal.

The multi-class classification benefits more because `PassFlag` eliminates the boundary entirely; class 0 versus class 1 accounts for the majority of errors in the base model, and `PassFlag` gives the model a direct correctness signal for resolving that split. If the goal is maximum agreement with the constructed label, the five feature Random Forest is the clear choice. If the goal is to have a more defensible evaluator of feedback quality that generalizes beyond the correctness features already embedded in the label definition, then the three feature Random Forest is the more meaningful result. An evaluator that captures qualitative differences among passing responses is more useful than one that mainly reproduces `PassFlag`.

VII. DISCUSSION AND LIMITATIONS

A. Discussion

The base and extended models answer different questions. The base models use only `PRelevance`, `HRevealingness`, and `CDistance`; they test whether the computed feedback-quality features carry useful signal on their own. The extended models add the binary repair-correctness value and `EditDistance`, both closely tied to the plausibility label; they therefore serve mainly as a comparison rather than as the primary evaluator. Similarly, the near-perfect five-feature Random Forest should not be treated as the main result. It mostly shows that the constructed label can be reconstructed when correctness-related fields are included.

The more useful result is that the three-feature Random Forest still performs well on the held-out problem using only the computed proxy features. However, these features do not contribute equally. `PRelevance` and `HRevealingness` show

limited separation between passing and failing PassFlag, while CDistance provides the strongest signal because it is tied to repair minimality. This supports the central operational claim of the paper: passing unit tests alone is insufficient for the constructed plausibility measure. A repair can be correct while the hint is vague, too revealing, or weakly connected to the student's bug. The computed features make this difference measurable as a screening signal, not as a replacement for human judgment.

B. Limitations

The main limitation is that the plausibility labels are derived from computed features, not from human ratings. Human evaluation is still the most reliable way to assess whether a hint actually helps a student learn. The labels used here should therefore be treated as operationalized plausibility labels, not as direct measurements of teaching quality.

The evaluation also uses only Problem 5 as the held-out test problem. This reduces problem-level leakage, but it does not prove that the evaluator will generalize to all introductory Python problems. Problem 5 has the fewest buggy submissions and a lower response-level pass rate than the overall dataset average, making it a compact but non-trivial held-out test problem.

The source dataset comes from a single introductory Python course, and the original hints, explanations, and repairs mostly reflect GPT-4 style outputs. This keeps the dataset consistent, but it limits the feedback style represented in the data.

CDistance is another limitation. It is based on the canonical minimal fix from Hints-in-Browser, so it favors repairs that stay close to that reference. A different correct repair may still be useful for teaching even if it is farther from the canonical repair. Some rows also do not have a valid CDistance value, so they cannot be scored.

Finally, PlausibilityA depends on fixed cutoffs. Small changes in PRelevance, HRevealingness, or CDistance can change the final label. PlausibilityB gives a more graded view, but it is still a constructed label and should be used for comparison rather than as an independent measure of feedback quality.

In particular, PRelevance and HRevealingness show limited separation between passing and failing repairs, so they should be treated as weak proxy checks rather than independent evidence of feedback quality.

VIII. Conclusions

This paper tests whether objective features from the IntroPyNUS dataset extended by Hints-in-Browser can provide a reproducible screening signal for LLM-generated debugging feedback. The results show that the constructed labels are learnable on a held-out problem, and that the three feedback-quality features carry useful signal even without directly using the binary repair-correctness value. That said, it does not claim to solve feedback evaluation. The three-feature Random Forest, trained on Problems 1-4 and tested on held-out Problem 5, predicts the four-class PlausibilityB label with 86.3% accuracy and 0.876 macro F1 using only PRelevance, HRevealingness, and CDistance. This result is more informative than the near-perfect five-feature Random Forest, which largely reconstructs the constructed label once PassFlag and EditDistance are included.

The four objectives set out in the introduction were each met. We restructured the nested Hints-in-Browser extension of IntroPyNUS into a flat, response-level table of 10,548 rows, which made per-response feature computation possible. On that table we defined three deterministic features. PRelevance, HRevealingness, and CDistance approximate feedback grounding, hint revealingness, and repair minimality, with no model judgment entering their computation. These features supported two labeling schemes: a strict binary PlausibilityA and a four-class PlausibilityB built from training-derived quantile buckets. We trained evaluators to compare feedback-quality features against repair-correctness features as an ablation.

The key takeaway is that a repair passing the tests should not automatically be treated as good tutoring feedback. Correctness is necessary, but it does not show whether the hint is grounded in the student's bug, whether it gives away too much of the repair, or whether the fix stays close to the student's original attempt. The evaluator should therefore be treated as a first-pass screening tool for large collections of generated feedback, not as a replacement for human rating or student-based evaluation.

References

[1] T. Phung, V.-A. Pădurean, J. Cambronero, S. Gulwani, T. Kohn, R. Majumdar, A. Singla, and G. Soares, "Generative AI for programming education: Benchmarking ChatGPT, GPT-4, and human tutors," Aug. 2023, arXiv:2306.17156.

[2] T. Phung, V.-A. Pădurean, A. Singh, C. Brooks, J. Cambronero, S. Gulwani, A. Singla, and G. Soares, "Automating human tutor-style programming feedback: Leveraging GPT-4 tutor model for hint generation and GPT-3.5 student model for hint validation," in *Proc. 14th Learning Analytics and Knowledge Conf. (LAK)*, 2024, pp. 12–23.

[3] N. Kotalwar, A. Gotovos, and A. Singla, "Hints-In-Browser: Benchmarking language models for programming feedback generation," in *NeurIPS 2024 Datasets and Benchmarks Track*, 2024.

[4] Y. Hu, U. Z. Ahmed, S. Mechtaev, B. Leong, and A. Roychoudhury, "Refactoring based program repair applied to programming assignments," in *Proc. ASE*, 2019, pp. 388–398.

[5] K. Chehbouni, M. Haddou, J. C. K. Cheung, and G. Farnadi, "Neither valid nor reliable? Investigating the use of LLMs as judges," Aug. 2025, arXiv:2508.18076.

[6] M. Chen *et al.*, "Evaluating large language models trained on code," Jul. 2021, arXiv:2107.03374.

[7] J. Austin *et al.*, "Program synthesis with large language models," Aug. 2021, arXiv:2108.07732.

[8] T. Phung, J. Cambronero, S. Gulwani, T. Kohn, R. Majumdar, A. Singla, and G. Soares, "Generating high-precision feedback for programming syntax errors using large language models," in *Proc. EDM*, 2023.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

[10] C. Koutcheme, N. Dainese, S. Sarsa, J. Leinonen, A. Hellas, and P. Denny, "Benchmarking educational program repair," in *NeurIPS'23 Workshop on Generative AI for Education*, 2023.

[11] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proc. EMNLP*, 2023, pp. 2511–2522.

[12] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, 2023.